

WordPress → Static HTML

Command guide for converting All-in-One WP Migration backups into deployable static sites.

For whoever runs conversions on this machine · generated from `docs/user-guide.html`

Quick start

The web interface

```
.venv\Scripts\python.exe app.py
```

Then open `http://127.0.0.1:8000` . Pick a backup, or tick several and press **Convert N backups**. The **Calculate time** button estimates a batch before you commit to it.

One site from the terminal

```
.venv\Scripts\python.exe convert.py C:\bkp\site.wpress -o C:\wpssc-jobs\exports
```

Every backup in a folder

```
.venv\Scripts\python.exe run_batch.py C:\bkp -o C:\wpssc-jobs\exports
```

It sizes itself from this machine's cores, memory and free disk, runs several conversions at once, refills each slot as a site finishes, and writes `summary.csv` beside the ZIPs.

Where everything goes

What	Where
Finished ZIPs	<code>C:\wpssc-jobs\exports\<site>-static-<date>-<time>.zip</code>
Job workspace	<code>C:\wpssc-jobs\<job-id>\</code>
Report	<code><job-id>\report\conversion-report.html</code>
Log	<code><job-id>\logs\job-<job-id>.log</code>
Screenshots and diffs	<code><job-id>\screenshots\</code>
Batch summary	<code><exports>\summary.csv</code>

The ZIP is hard-linked into `exports\` , so it costs no extra disk and survives deleting the workspace. Each run's date and time are in the name, so converting the same backup twice never overwrites the first result.

Batch options

Option	What it does
<code>--parallel N</code>	Conversions at once. Default: measured from the machine.
<code>--pages N</code>	Pages each conversion renders at once. Default: the machine's page budget divided between them.
<code>--retry</code>	Convert sites that already have a ZIP again.
<code>--no-resume</code>	Start interrupted sites over rather than continuing them.
<code>--screenshots</code>	Capture and compare screenshots against the original.
<code>--flat</code>	One file per page (<code>about.html</code>) instead of a folder per page.
<code>--file-links</code>	Write links as <code>contact-us/index.html</code> rather than <code>contact-us/</code> .
<code>--jobs-dir PATH</code>	Where job workspaces are written. Needs roughly 3× the backup size per running job.

Stopping and continuing

Stopping

From the interface, **Cancel** stops one conversion at its next checkpoint; the others carry on. **Cancel remaining** stops everything not yet finished. Nothing is deleted.

From the terminal, Ctrl+C stops the whole batch, including the conversions running at that moment.

Continuing a stopped batch

```
.venv\Scripts\python.exe run_batch.py C:\bkc -o C:\wpsc-jobs\exports
```

Run the same command again. Sites with a ZIP are skipped; sites that were interrupted **continue where they stopped**, skipping extraction and the database import — about half an hour each on a 3 GB backup. Everything else converts normally.

```
20 backup(s); 16 to convert, 4 already done
4 stopped part-way and will continue where they left off
```

Continuing one site

```
.venv\Scripts\python.exe convert.py --resume <job-id> --rerender --resume-port <port> -o C:\wpsc-jobs\exports
```

The job id and port are in that job's log, on the line reading `WordPress is serving on http://127.0.0.1:<port>`. The port matters because captured pages hold absolute URLs to the port they were rendered against.

A site that failed is not resumed automatically. It stopped because something went wrong, and reusing the half-built result is unlikely to give a different answer, so the batch starts it over. Only interrupted work — cancelled, or still running when the process stopped — is continued.

Starting over instead of continuing

The same command behaves differently because it reads what is on disk, not because it remembers anything. Two flags override that.

Command	What it does
<code>--no-resume</code>	Interrupted sites start from the beginning. Sites that already have a ZIP are still skipped.
<code>--retry</code>	Sites that already have a ZIP are converted again. Implies <code>--no-resume</code> , because redoing finished work and continuing old work are contradictory.
<code>--retry --no-resume</code>	A complete restart: every backup in the folder converted from scratch, nothing skipped, nothing reused.

Redo the interrupted sites properly, keep the finished ones:

```
.env\Scripts\python.exe run_batch.py C:\bcp -o C:\wpsc-jobs\exports --no-resume
```

Convert everything again, from the beginning:

```
.env\Scripts\python.exe run_batch.py C:\bcp -o C:\wpsc-jobs\exports --retry --no-resume
```

Redo one site only: delete its ZIP from `exports\` and run the ordinary command. With no ZIP and no workspace it converts from scratch; with a workspace still present it would resume, so delete that too if you want it truly fresh.

```
del C:\wpsc-jobs\exports\site-static-*.zip
rmdir /s /q C:\wpsc-jobs\<job-id>
.env\Scripts\python.exe run_batch.py C:\bcp -o C:\wpsc-jobs\exports
```

Prefer a complete restart after a fix to the restore. A resume reuses the database and the extracted files, so a change to how the site is restored — a plugin setting, a theme repair — only takes effect on a fresh conversion. A change to rendering or to the export is picked up by a resume, because those stages run again either way.

Reading the result

`summary.csv` has one row per site: pages, problems, source issues, visual score, minutes and ZIP path. Sort by `problems` and look only at the rows that need you.

In the report	Means
Problems	Faults in the export. These need attention.
Source issues	Faults in the original site that the export reproduces faithfully — broken images, dead links. Not your problem to fix here.
Case mismatches	A link whose capitalisation differs from the file's. Works on Windows, 404s on a Linux host.
Visual similarity	Sampled pages compared against the original. Below ~95% on Linux usually means missing fonts.
Needs a server	Forms, search, comments and AJAX. Preserved visually; they cannot function without PHP.

Checking the export yourself

```
cd C:\wpsc-jobs\<job-id>\output
python -m http.server 8080
```

Then open `http://localhost:8080/`. Open the site this way, not by double-clicking `index.html` — browsers apply much stricter rules to `file://` pages, and a perfectly good export can look broken.

Troubleshooting

Symptom	Cause and fix
"not enough free space"	A conversion needs about 3× the backup. Free space, or point <code>--jobs-dir</code> at a bigger disk.
A job fails part-way with a disk error	Free space fell below what the run needed. The batch checks up front, but other programs consume disk too.
Pages look unstyled	Open the site over HTTP, not <code>file://</code> . If it persists, check the report for missing stylesheets.
Conversion is slow	Close other heavy programs. On a 2-core machine an editor and a browser are direct competition for the conversion.
Dependencies missing	<code>.venv\Scripts\python.exe scripts\doctor.py</code> checks PHP, MariaDB and Chromium and says what to install.

On a Linux server

```
sudo ./setup.sh
.venv/bin/python scripts/doctor.py
.venv/bin/python -m pytest -q -m integration
.venv/bin/python run_batch.py /srv/backups -o /srv/exports
```

Run it under `tmux` so closing the terminal cannot stop it. Ubuntu 24.04 is required — 22.04 ships Python 3.10 and cannot run this. Full detail, including the verification checklist, is in `docs/LINUX.md`.

Convert one site before trusting a batch of twenty. Read its report and open the exported site. Twenty sites converted wrongly is twenty sites to convert again.

Every conversion restores the backup into a temporary, loopback-only WordPress, renders each page in a real browser, and destroys the temporary site afterwards. Nothing is uploaded, and the original backup is never modified.